

Распределения

Для начала загрузим датасет каунтов в [ReReq2](#) А также активируем R пакеты

```
In [ ]: # Загрузка R magic
%load_ext rpy2.ipython

# Установка нужных пакетов
# !Rscript -e 'install.packages("fitdistrplus")'
```

The rpy2.ipython extension is already loaded. To reload it, use:
%reload_ext rpy2.ipython

Принципы работы с R из колаба

Для написания кода на R необходимо в начале ячейки прописать `%%R`

А затем писать код на R

```
In [ ]: %%R

a ← c(1,2,3,4,5,6)
b ← a^2
b
```

```
[1] 1 4 9 16 25 36
```

Переменные доступны между ячейками

```
In [ ]: %%R
c ← b - 2
c
```

```
[1] -1 2 7 14 23 34
```

Можно импортировать переменные из питона в R. При помощи флага `%%R -i python_var_name`

```
In [ ]: # Python
x = 15
```

```
In [ ]: %%R -i x
c + x
```

```
[1] 14 17 22 29 38 49
```

А для передачи из R в питон есть флаг `%%R -o R_var`

```
In [ ]: %%R -o c
c
```

```
[1] -1 2 7 14 23 34
```

```
In [ ]: c
```

```
Out[ ]: array([-1., 2., 7., 14., 23., 34.])
```

Вернемся к распределениям

Импорты и настройки

```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Настройка отображения
plt.rcParams["figure.dpi"] = 100
```

Чтение датасета с каунтами

```
In [ ]: # Прочтение датасета
df = pd.read_csv("LUAD_counts.tsv", sep="\t")
df.index = [gene.split(".")[0] for gene in df.gene_name]
del df["gene_name"]
df = df.T
df.head()
```

```
Out[ ]:
```

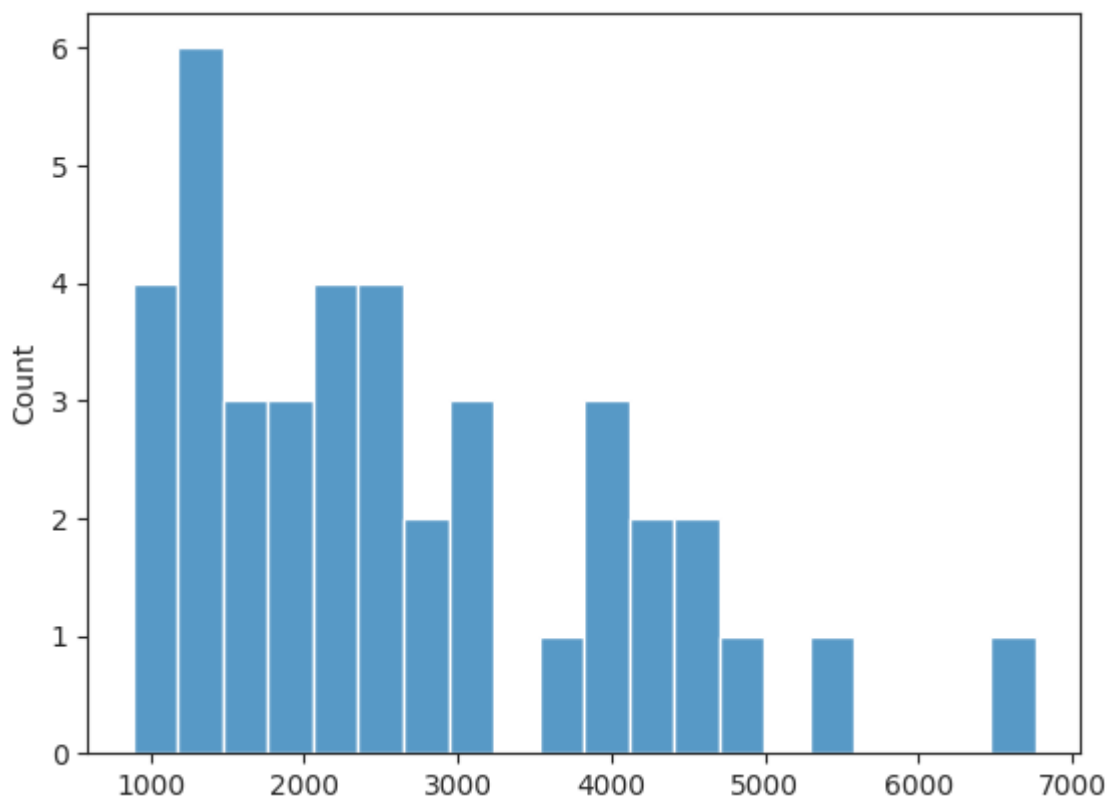
	TSPAN6	TNMD	DPM1	SCYL3	C1orf112	FGR	CFH	FUCA2	GCLC	NFYA	...	ACTL10	AC1197
sample_0	4275	0	1071	630	542	372	10445	6760	31744	2022	...	0	
sample_1	1536	0	765	510	172	911	4290	1097	807	1043	...	0	
sample_2	5009	13	2731	919	321	2299	9368	3812	2704	2496	...	0	
sample_3	1889	0	1941	548	301	879	5994	2310	2552	1332	...	0	
sample_4	1329	0	1395	297	320	387	6215	3142	685	1676	...	0	

5 rows × 19962 columns

Посмотрим на распределение глазами

```
In [ ]: gene = "FUCA2"
expr = np.array(df[gene]) # Вектор экспрессий гена C01
sns.histplot(expr, bins=20)
```

```
Out[ ]: <Axes: ylabel='Count'>
```



Проверим данные на **нормальность** методом **Критерий Шапиро-Уилка**

Здесь нулевая гипотеза - данные распределены нормально => низкий p-value - данные ненормальные

```
In [ ]: from scipy.stats import shapiro

stat, p = shapiro(df[gene])
print(f"Statistics={stat:3f}, p={p:3f}")

Statistics=0.914307, p=0.005122
```

График Дисперсии от среднего

```
In [ ]: means = np.mean(df.values, axis=0) # Среднее каунтов каждого гена
vars = np.var(df.values, axis=0) # Дисперсии каунтов каждого гена

means = means[vars > 0] # Фильтруем, чтобы дисперсия и средние были больше 0
vars = vars[vars > 0] # Если var > 0, то и mean > 0

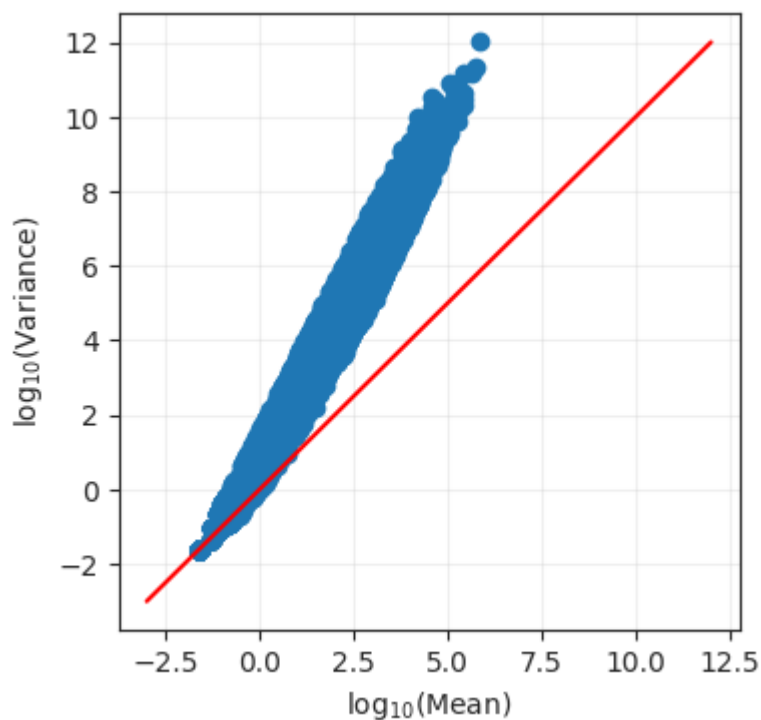
means = np.log10(means)
vars = np.log10(vars)
```

```
In [ ]: sns.set_style("ticks")
fig, ax = plt.subplots(figsize=(4, 4), dpi=100)

ax.scatter(means, vars)
ax.plot([-3, 12], [-3, 12], color="red")
ax.grid(alpha=0.3)

ax.set_xlabel("log$_{10}$(Mean)")
ax.set_ylabel("log$_{10}$(Variance)")
```

```
Out[ ]: Text(0, 0.5, 'log$_{10}$(Variance)')
```



Видно, что Пуассон не подходит

(только если для низкоэкспрессированных)

Тестирование распределений

Импортируем вектор экспрессий гена в R

А также загрузим библиотеку, при помощи которой сможем подобрать максимально правдоподобные распределения данных.

Т.е. предположим данные нормальные, тогда вопрос - Какие параметры у распределения

```
In [ ]: %%R -i expr
library(fitdistrplus)
```

Импортированный expr в R имеет тип данных - лист, но R Лучше работать с векторами

```
In [ ]: %%R
data <- as.vector(expr)
```

Подбираем параметры нормального распределения, которое могло бы описывать данные. А также определяем likelihood, что это действительно нормальное распределение. Критерий AIC - likelihood нормированные как кол-во параметров распределения.

AIC – Информационный критерий Акаике

$$AIC = 2k - 2(\text{LogLikelihood})$$

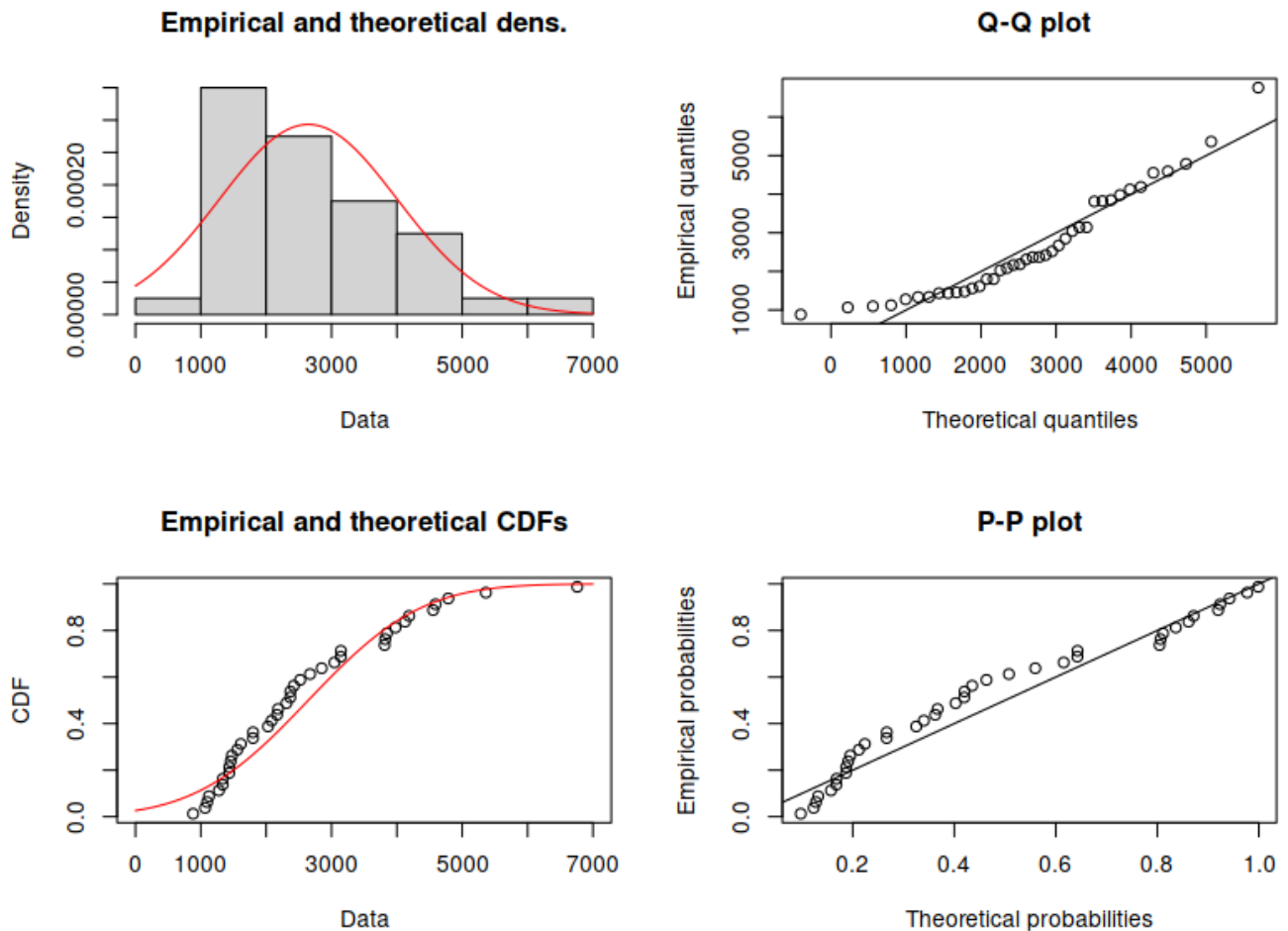
k – кол-во параметров модели

Чем AIC меньше, тем лучше распределение описывает данные

```
In [ ]: %%R
fit.norm <- fitdist(data, "norm")
summary(fit.norm)
```

```
Fitting of the distribution ' norm ' by maximum likelihood
Parameters :
      estimate Std. Error
mean 2645.300    215.1633
sd   1360.424    152.0934
Loglikelihood: -345.3796    AIC:  694.7592    BIC:  698.137
Correlation matrix:
      mean      sd
mean  1.0000000000 -0.0002325244
sd   -0.0002325244  1.0000000000
```

```
In [ ]: %%R -r 100 -w 800 -h 600
plot(fit.norm)
```



Графики

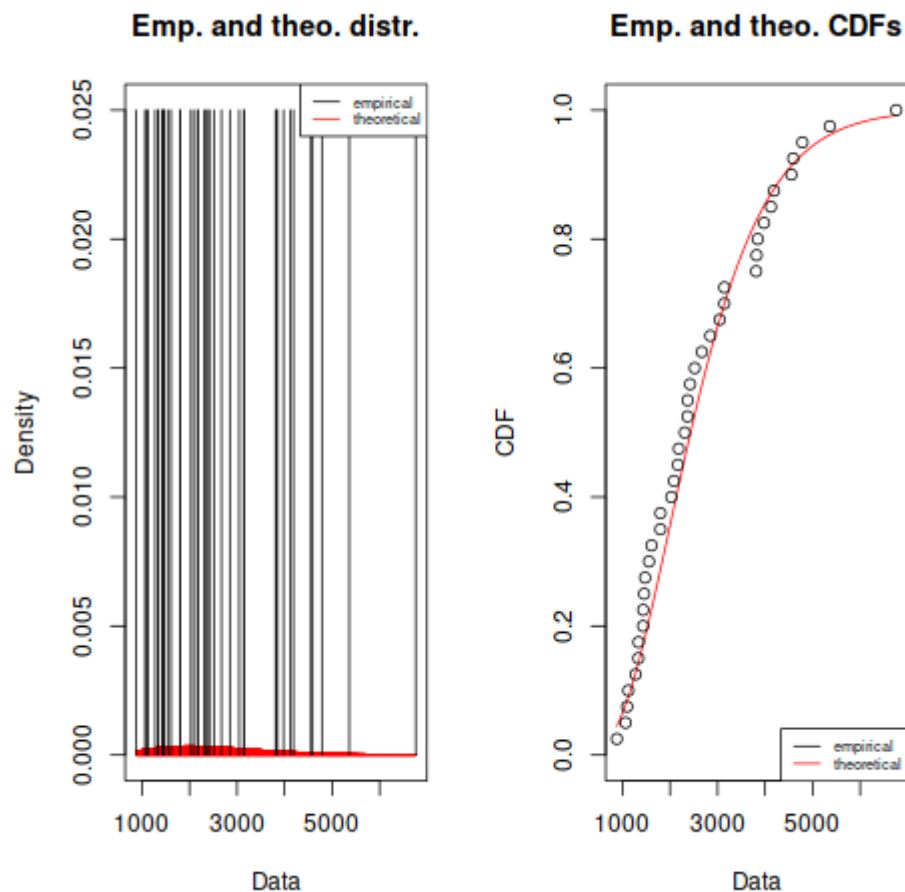
Empirical & Theoretical CDF's

По сути график плотности вероятности распределения (красный) И точки, обозначающие данные

В идеале должно быть почти 100% наложения

Попробуем др распределения

```
In [ ]: %%R
fit.nb ← fitdist(data, "nbinom")
summary(fit.nb)
plot(fit.nb)
```



```
In [ ]: %%R
fit.lnorm ← fitdist(data, "lnorm")
summary(fit.lnorm)
```

Fitting of the distribution ' lnorm ' by maximum likelihood
Parameters :

	estimate	Std. Error
meanlog	7.7528776	0.08014155
sdlog	0.5068596	0.05666764

Loglikelihood: -339.6918 AIC: 683.3836 BIC: 686.7614

Correlation matrix:

	meanlog	sdlog
meanlog	1	0
sdlog	0	1

```
In [ ]: %%R
fit.pois ← fitdist(data, "pois")
summary(fit.pois)
```

Fitting of the distribution ' pois ' by maximum likelihood
Parameters :

	estimate	Std. Error
lambda	2645.3	8.132284

Loglikelihood: -13354.38 AIC: 26710.76 BIC: 26712.45

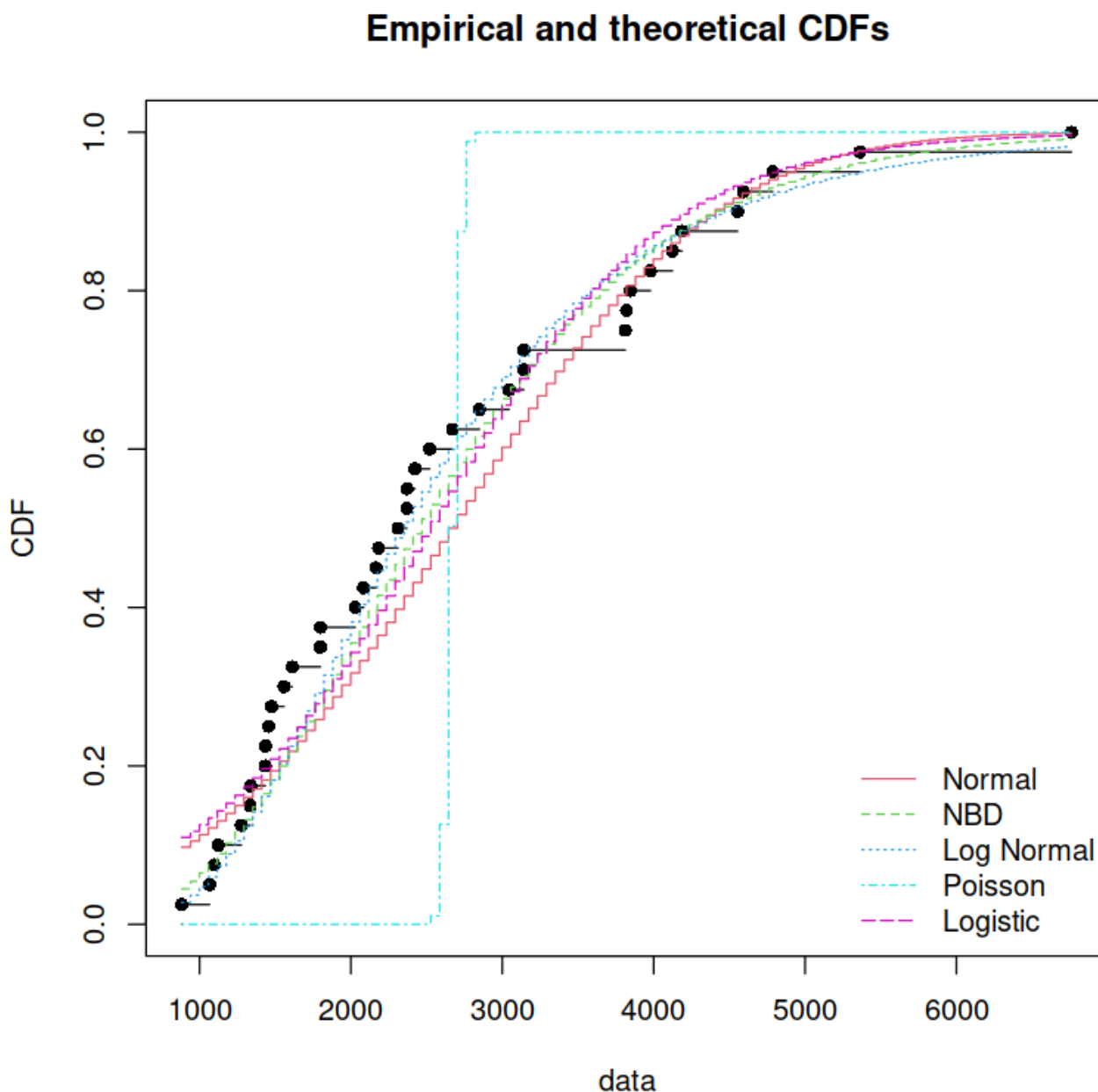
```
In [ ]: %%R
fit.logis ← fitdist(data, "logis")
summary(fit.logis)
```

```
Fitting of the distribution 'logis' by maximum likelihood
Parameters :
      estimate Std. Error
location 2500.8548    215.4420
scale     773.7669    101.1352
Loglikelihood: -345.555    AIC: 695.1101    BIC: 698.4878
Correlation matrix:
      location      scale
location 1.00000000 0.09542037
scale    0.09542037 1.00000000
```

Сравнение тестов

Теперь можно сравнить плотности вероятности опытного распределения и теоретически предполагаемых

```
In [ ]: %%R -r120 -h800 -w800
cdfcomp(list(fit.norm, fit.nb, fit.lnorm, fit.pois, fit.logis), legendtext = c("Norma
```



```
In [ ]: %%R
summary(fit.norm)
```

```
Fitting of the distribution ' norm ' by maximum likelihood
Parameters :
      estimate Std. Error
mean 2645.300    215.1633
sd   1360.424    152.0934
Loglikelihood: -345.3796   AIC:  694.7592   BIC:  698.137
Correlation matrix:
              mean          sd
mean  1.000000000000 -0.0002325244
sd    -0.0002325244  1.00000000000
```

In []:

```
%%R
summary(fit.nb)

Fitting of the distribution ' nbinom ' by maximum likelihood
Parameters :
      estimate Std. Error
size    4.080454    0.8791315
mu  2645.464718 207.2683347
Loglikelihood: -340.4042   AIC:  684.8084   BIC:  688.1862
Correlation matrix:
              size          mu
size  1.000000000000 -0.0001721982
mu    -0.0001721982  1.00000000000
```

In []:

```
%%R
summary(fit.lnorm)

Fitting of the distribution ' lnorm ' by maximum likelihood
Parameters :
      estimate Std. Error
meanlog 7.7528776 0.08014155
sdlog   0.5068596 0.05666764
Loglikelihood: -339.6918   AIC:  683.3836   BIC:  686.7614
Correlation matrix:
      meanlog sdlog
meanlog      1      0
sdlog        0      1
```

Видно, что AIC наибольший у NBD среди дискретных распределений (у lognorm - меньше, но оно непрерывное)

А также т.к. данных меньше

Бутстреп

Оценка стабильности данных

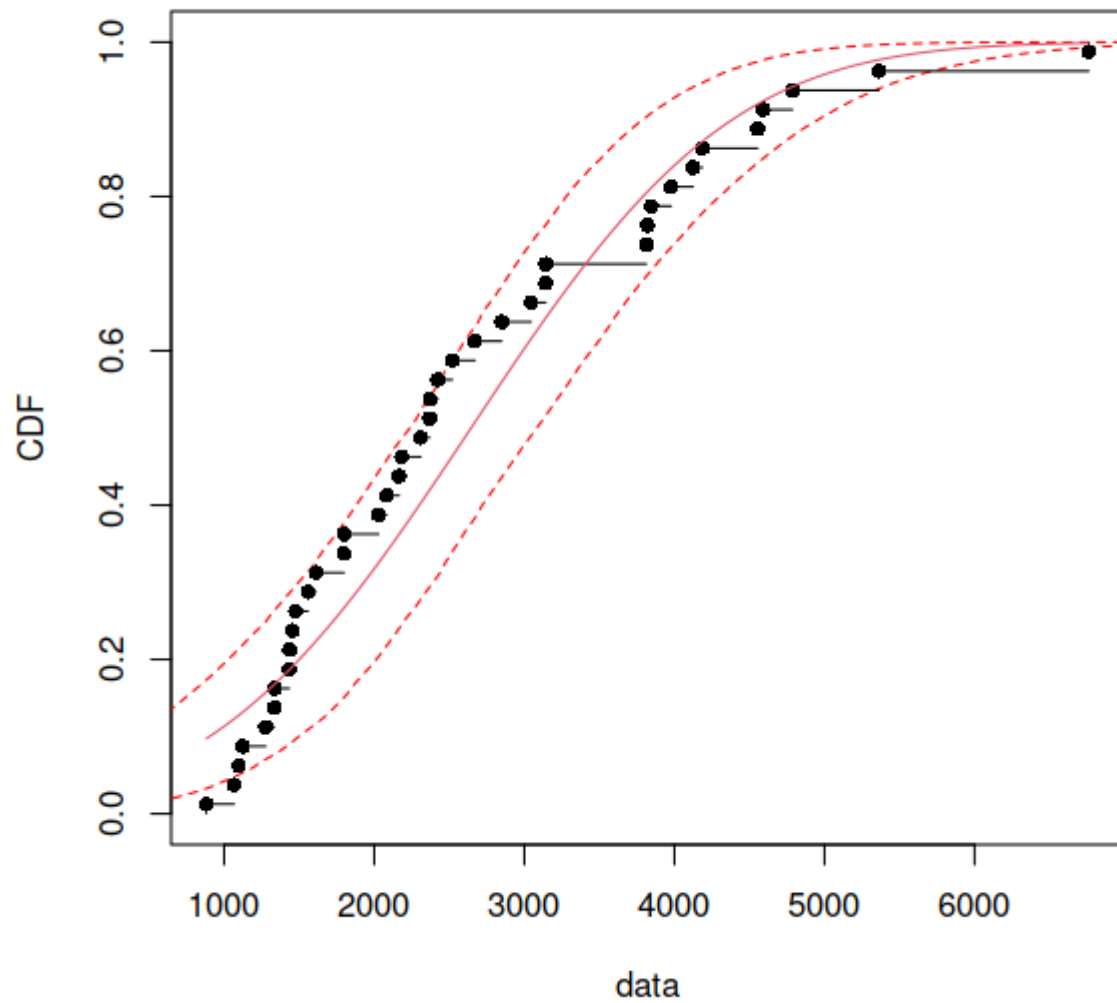
Normal (все плохо)

In []:

```
%%R -r 100 -w 600 -h 600

fit.norm.boot ← bootdist(fit.norm, 'param')
CIcdfplot(fit.norm.boot, CI.output = "probability")
```

Empirical and theoretical CDF with CI

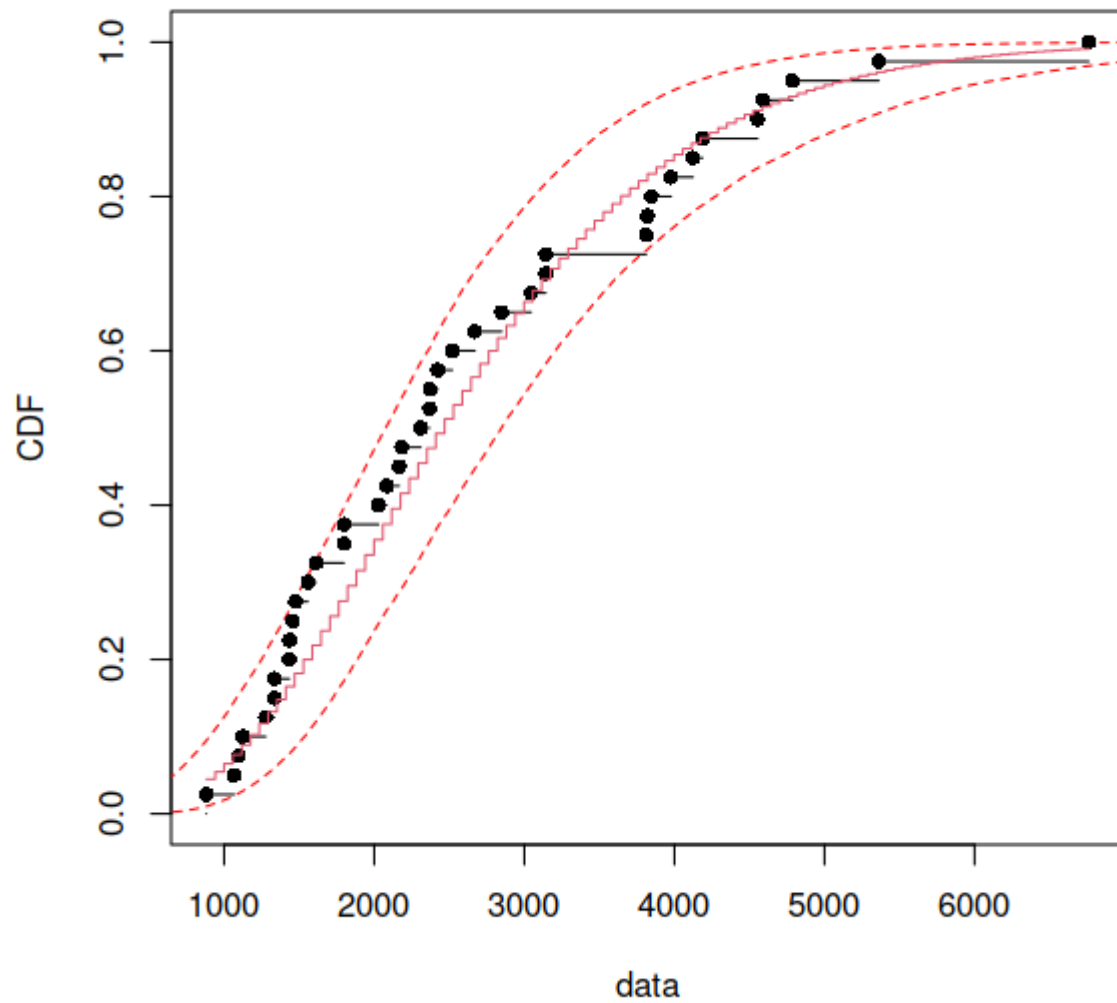


NBD (все точки в пределах доверительного интервала)

```
In [ ]: %%R -r 100 -w 600 -h 600
```

```
fit.nb.boot ← bootdist(fit.nb, 'param')  
Cicdfplot(fit.nb.boot, CI.output = "probability")
```

Empirical and theoretical CDF with CI

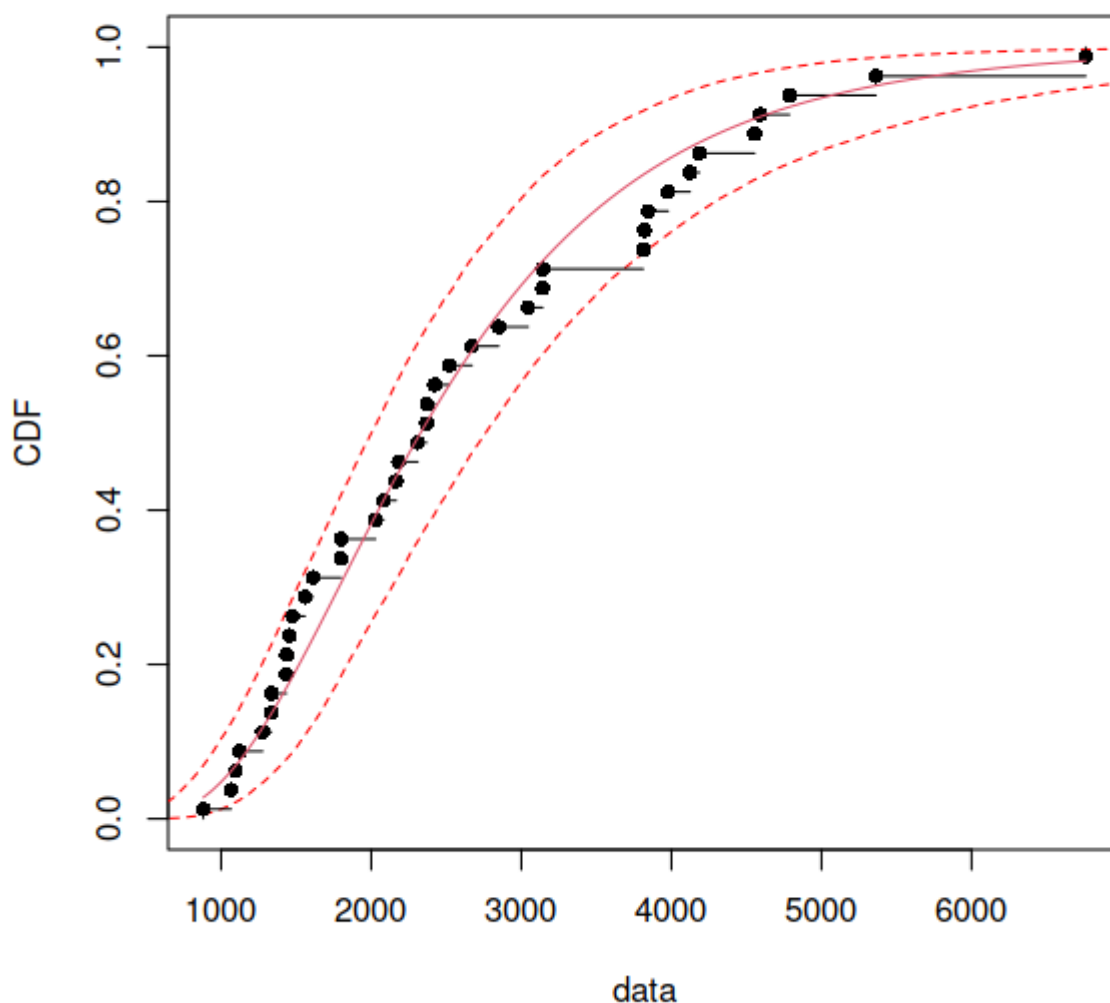


LogNormal (все точки в пределах доверительного интервала)

```
In [ ]: %%R -r 100 -w 600 -h 600
```

```
fit.lnorm.boot ← bootdist(fit.lnorm, 'param')  
Cicdfplot(fit.lnorm.boot, CI.output = "probability")
```

Empirical and theoretical CDF with CI



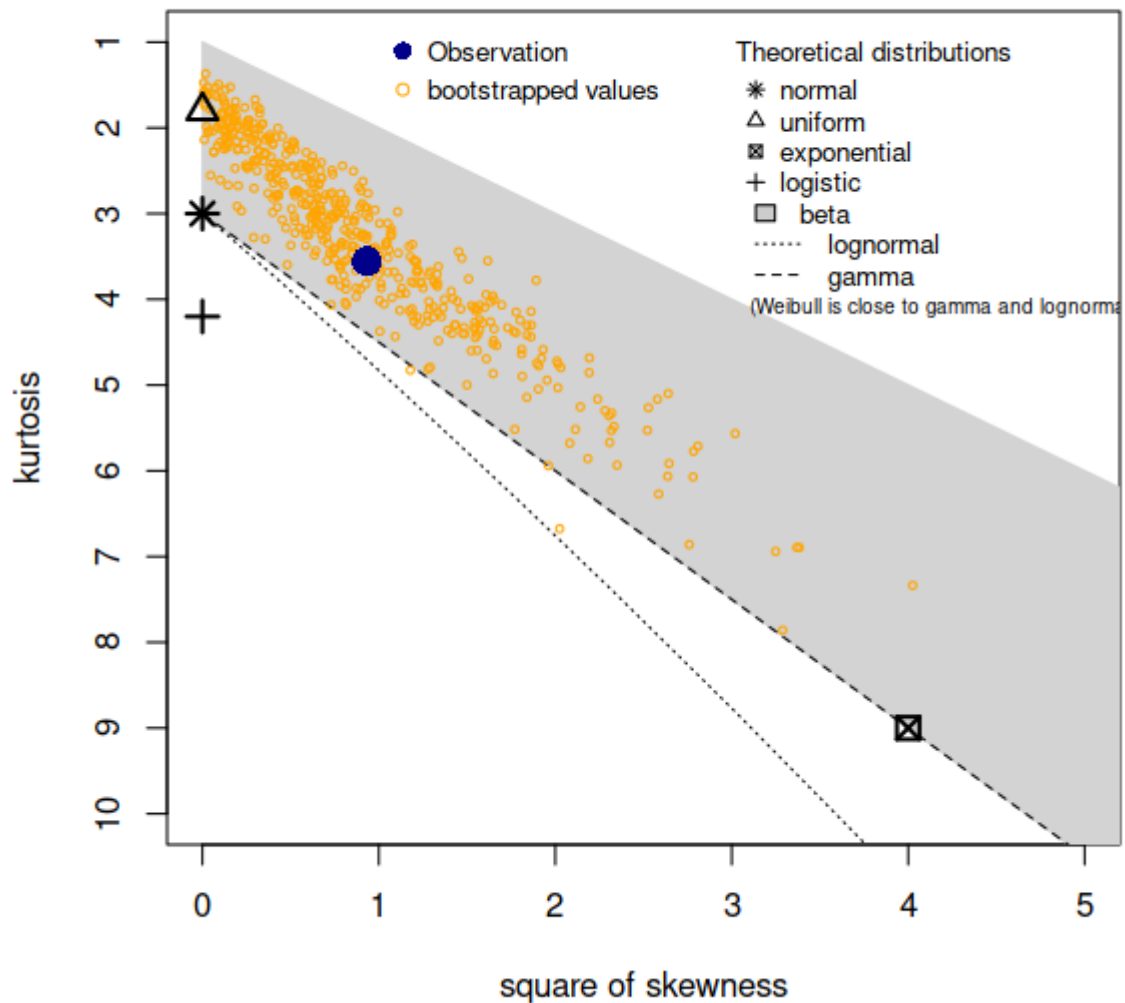
3ий и 4ий моменты

Можно использовать оценки 3 и 4 момента распределения, для прикидки распределения

```
In [ ]: %%R -r 100 -w 600 -h 600  
descdist(data, discrete=FALSE, boot=500)
```

```
summary statistics  
-----  
min: 882   max: 6760  
median: 2339.5  
mean: 2645.3  
estimated sd: 1377.755  
estimated skewness: 0.9652628  
estimated kurtosis: 3.554715
```

Cullen and Frey graph



Для каждого распределения существует область допустимых значений для моментов

Н.п.

- Для нормального распределения 3 и 4 моменты всегда в 1 точке (символ * на графике)
- Равномерного - Δ
- Бетта распределение - серое пространство
- и так далее

Датасет обозначен ● а бутстреп реплики ● (они необходимы для компенсации маленького размера датасета, если данных много то можно без него)

И тут уже logNorm, который был лучше всех в предыдущих оценках фитится довольно плохо

Но, тк. точно известно, что это дискретное распределение, то можно сразу отбросить все сплошные (`discrete=TRUE`)

```
In [ ]: %%R -r 100 -w 600 -h 600
descdist(data, discrete=TRUE, boot=500)
```

summary statistics

min: 882 max: 6760

median: 2339.5

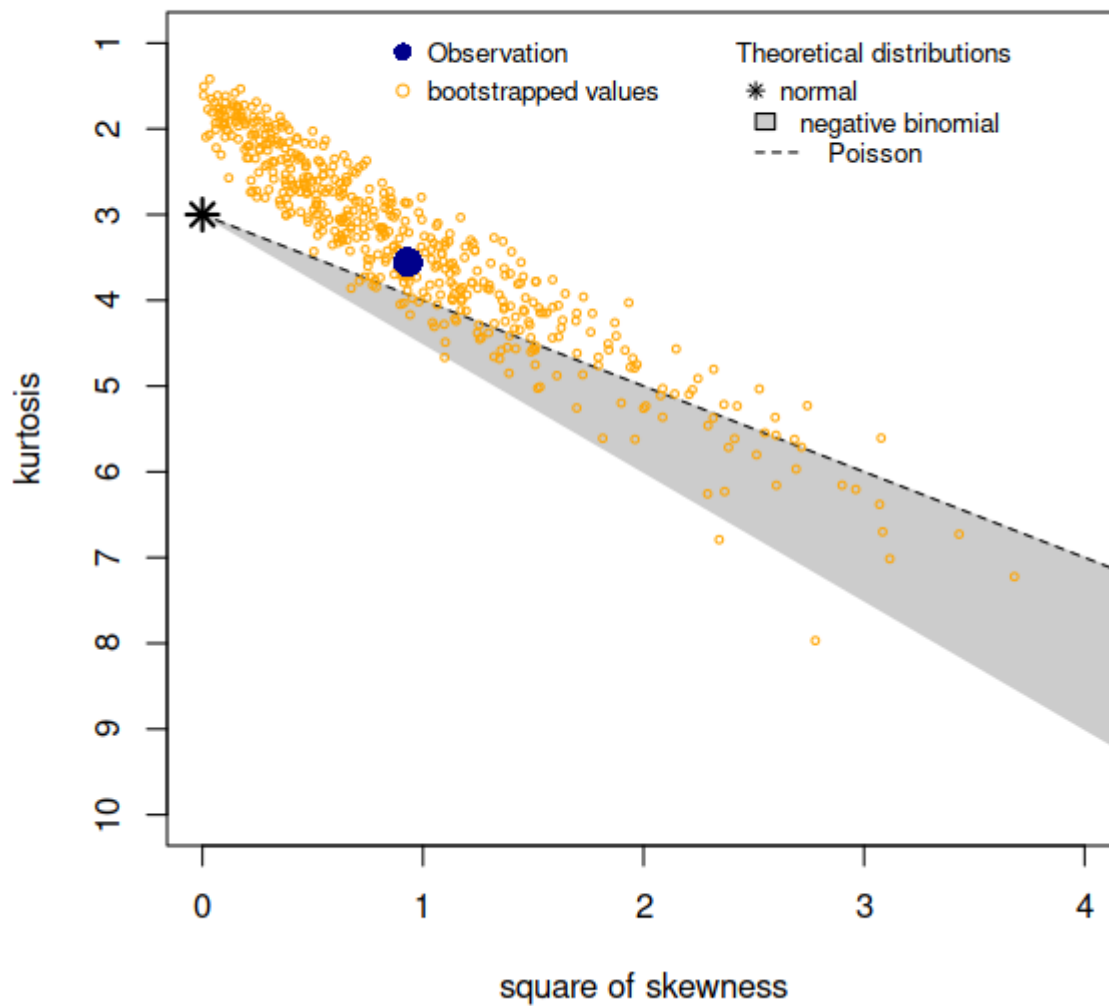
mean: 2645.3

estimated sd: 1377.755

estimated skewness: 0.9652628

estimated kurtosis: 3.554715

Cullen and Frey graph



И тут - NBD очень хорошо фитится

(Можно еще убрать бутстрэп, для прикидки без оценки стабильности)

In []: